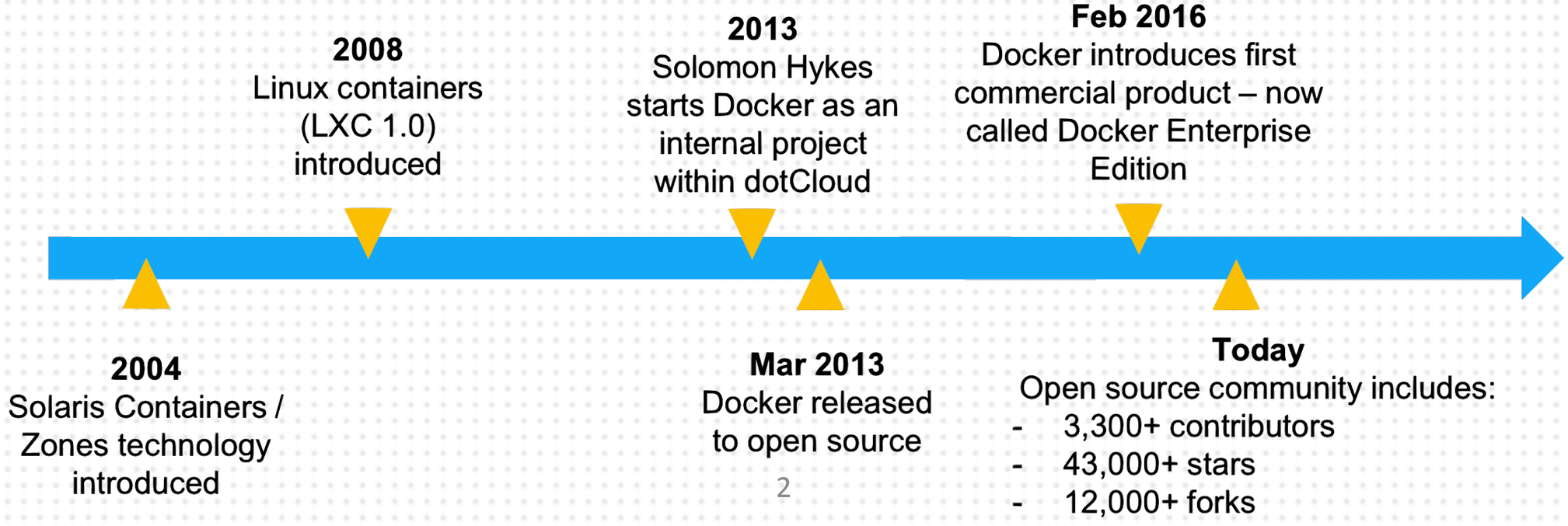
# Docker

Ehsan Khamespanah

Spring 2024

# What is Docker

- Docker is an open platform for developing, shipping, and running applications.
- Decker is designed to deliver you applications faster.
- With docker you can separate your application from your infrastructure and treat your infrastructure like a managed application.



# Incredible adoption in 4 Years!



14M

Docker Hosts



900K

Docker apps



77K%

Growth in Docker job listings



12B

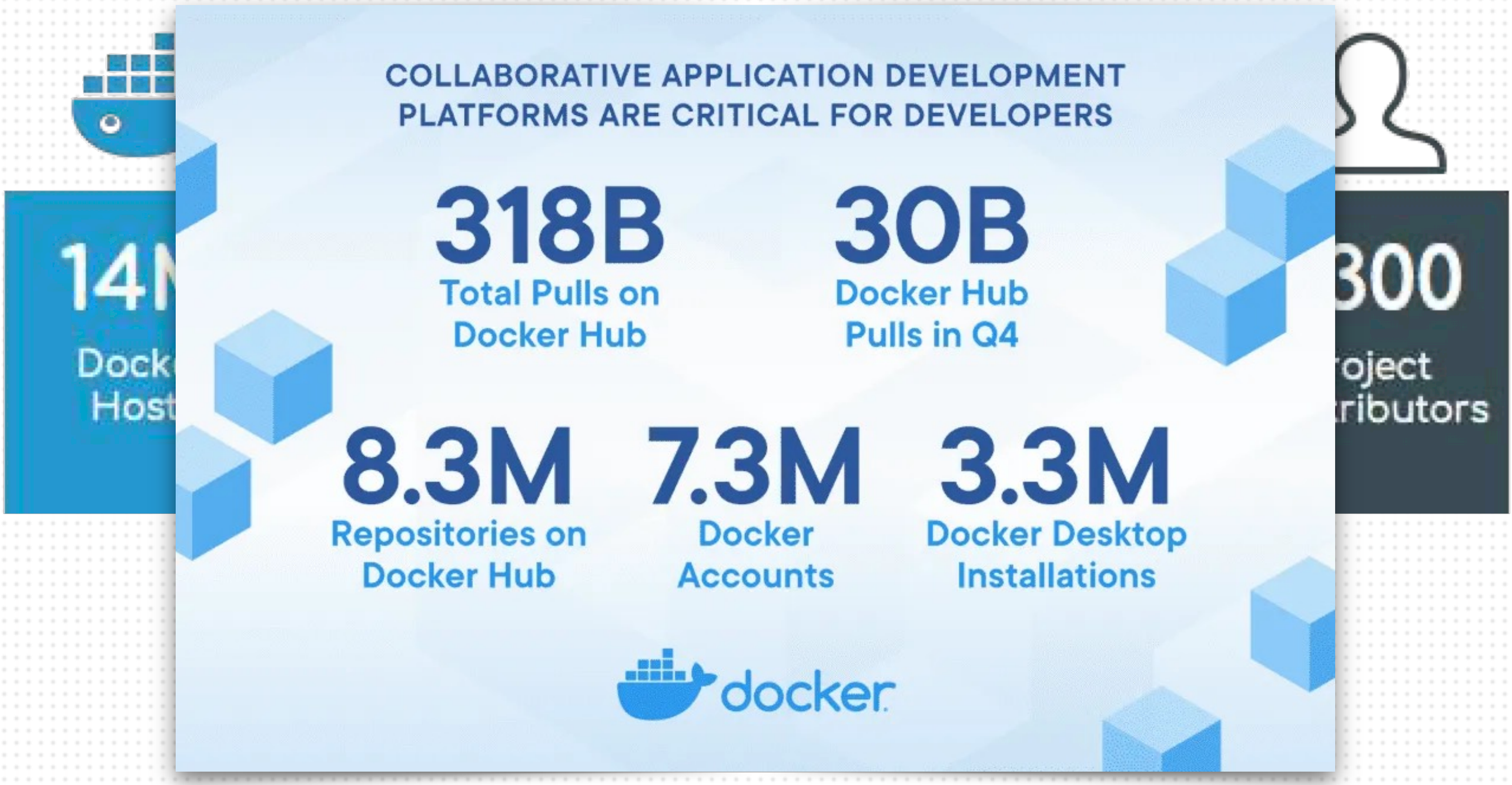
Image pulls Over 390K% Growth



3300

Project Contributors

# Incredible adoption in 4 Years!



# Docker Family Tree




Open source **framework** for assembling core components that make a container platform

Intended for:
Open source contributors + ecosystem developers




Enterprise Edition

Subscription-based, commercially supported **products** for delivering a secure software supply chain

Intended for:
Production deployments + Enterprise customers

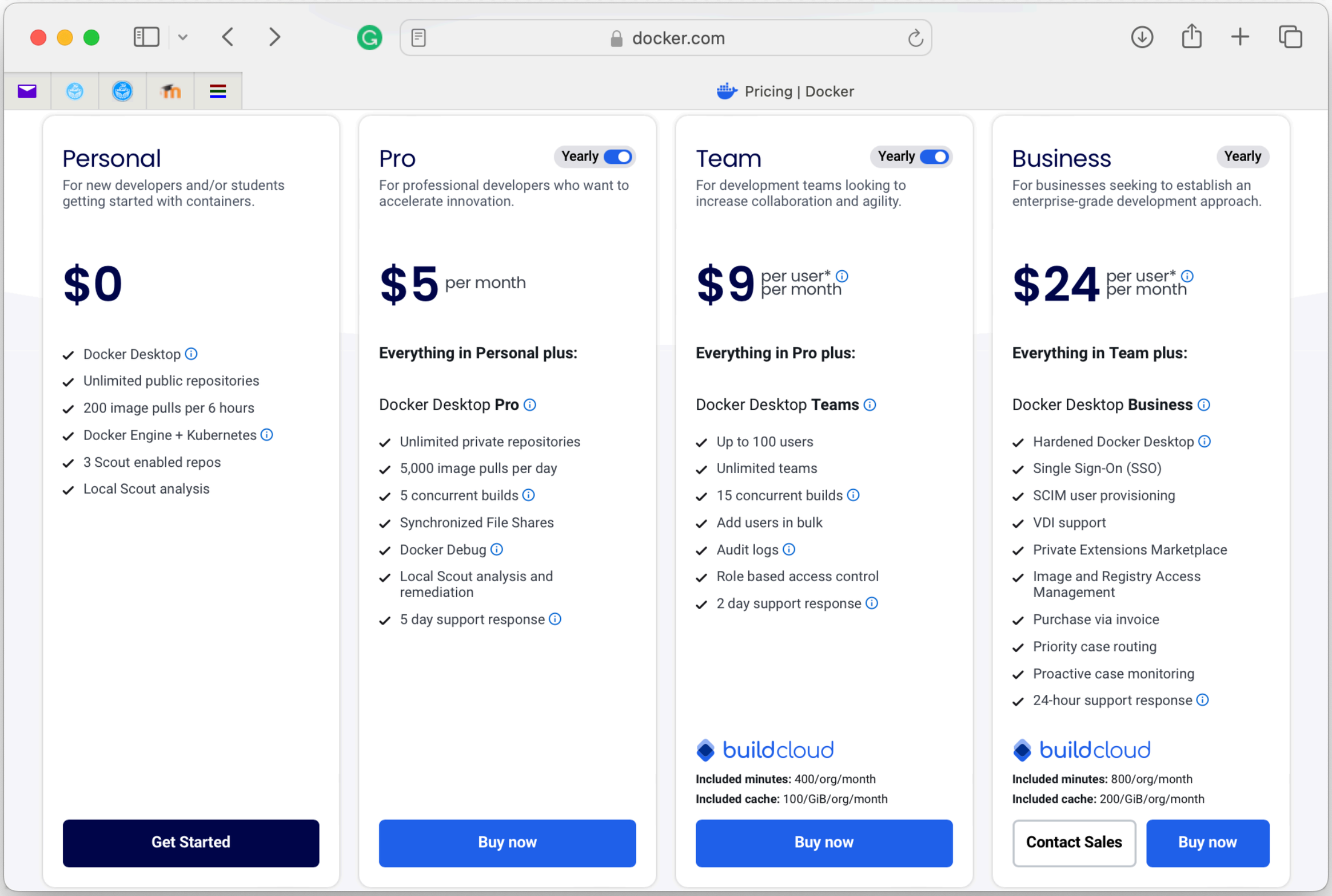



Community Edition

Free, community-supported **product** for delivering a container solution

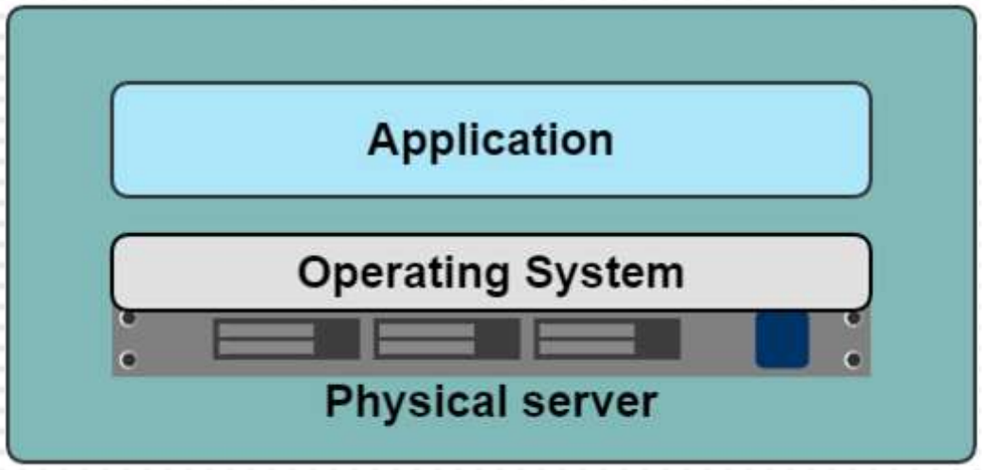
Intended for:
Software dev & test

# Docker Family Tree



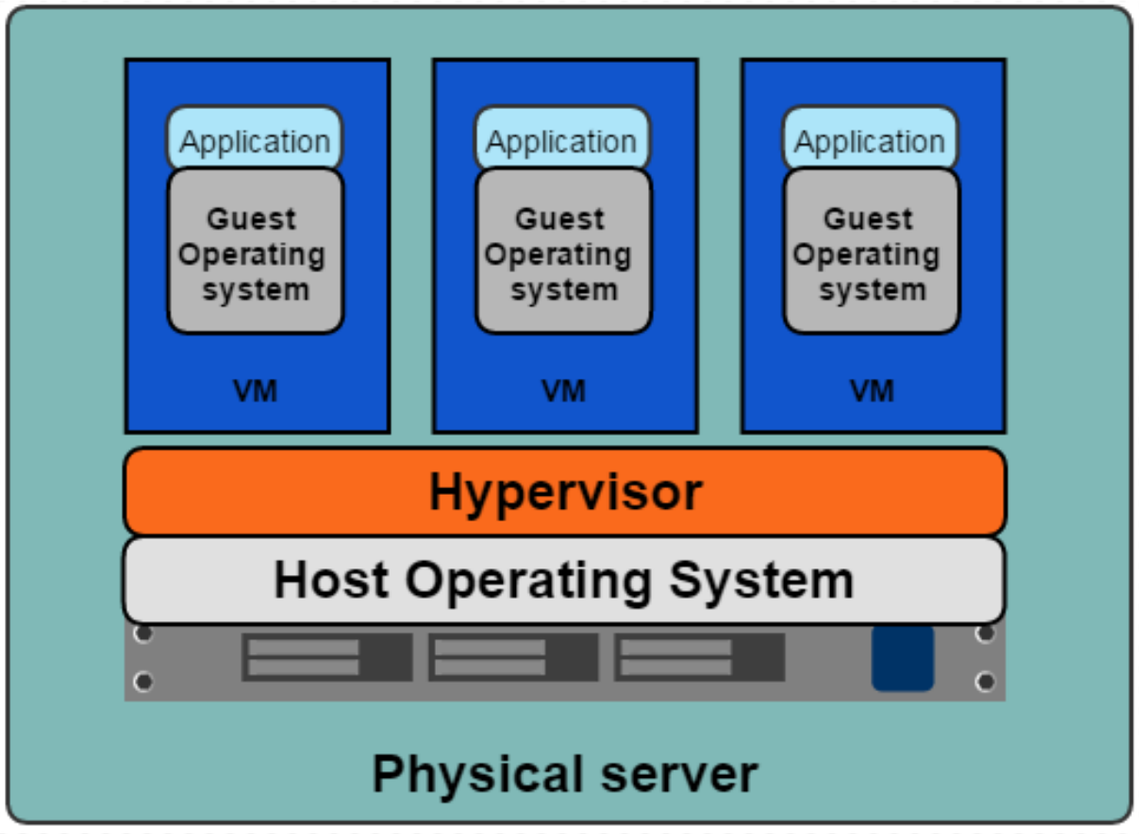
# How to Deploy Applications

## One App on One Server




- Slow deployment
- Huge cost
- Difficult to scale and migrate
- Vendor lock in

## Hypervisor-based Virtualization



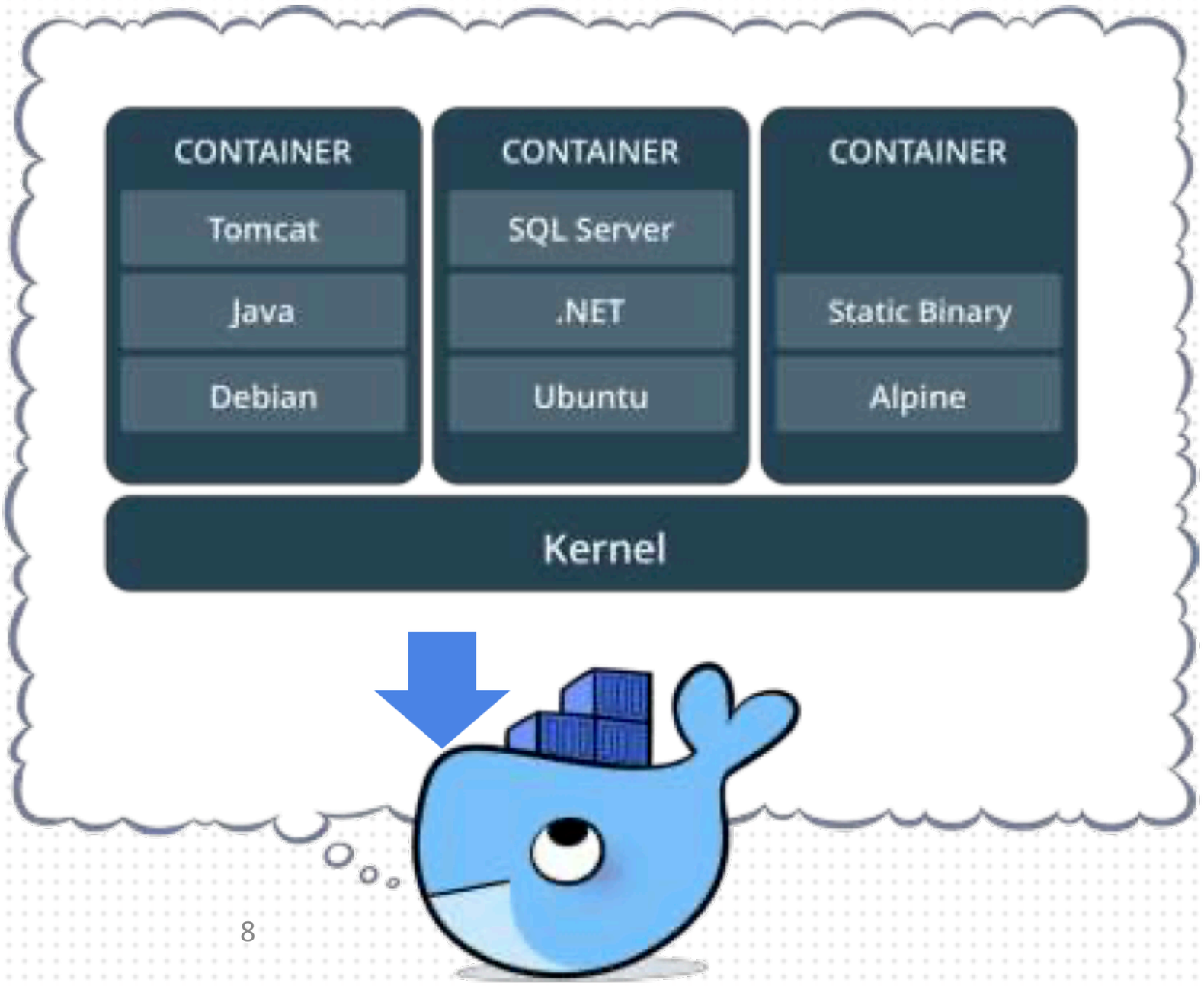

- Applications are run in virtual machines

# Limitation of Hypervisor-based Virtualization

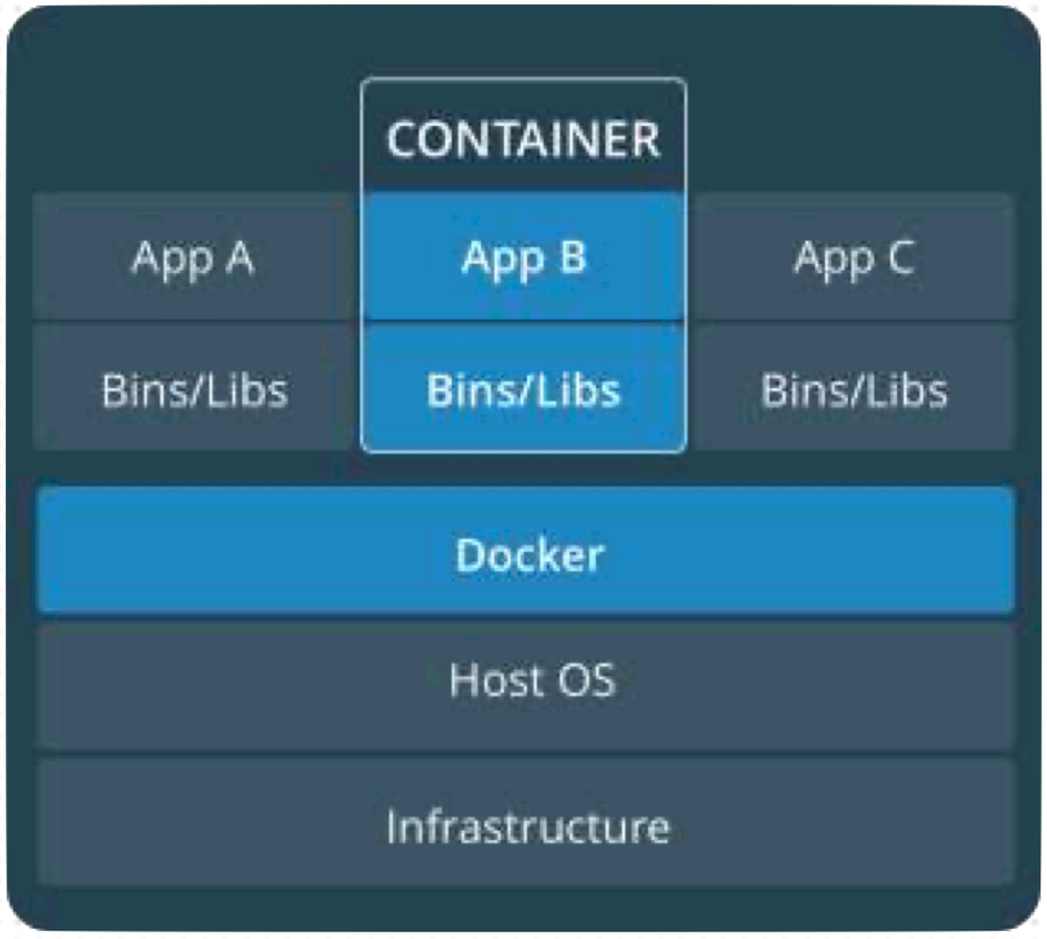
- Each VM stills requires
  - CPU allocation
  - Storage
  - RAM
  - An entire guest operating system
- The more VMs you run, the more resources you need
- Guest OS means wasted resources
- Application portability not guaranteed

# What is a Container?

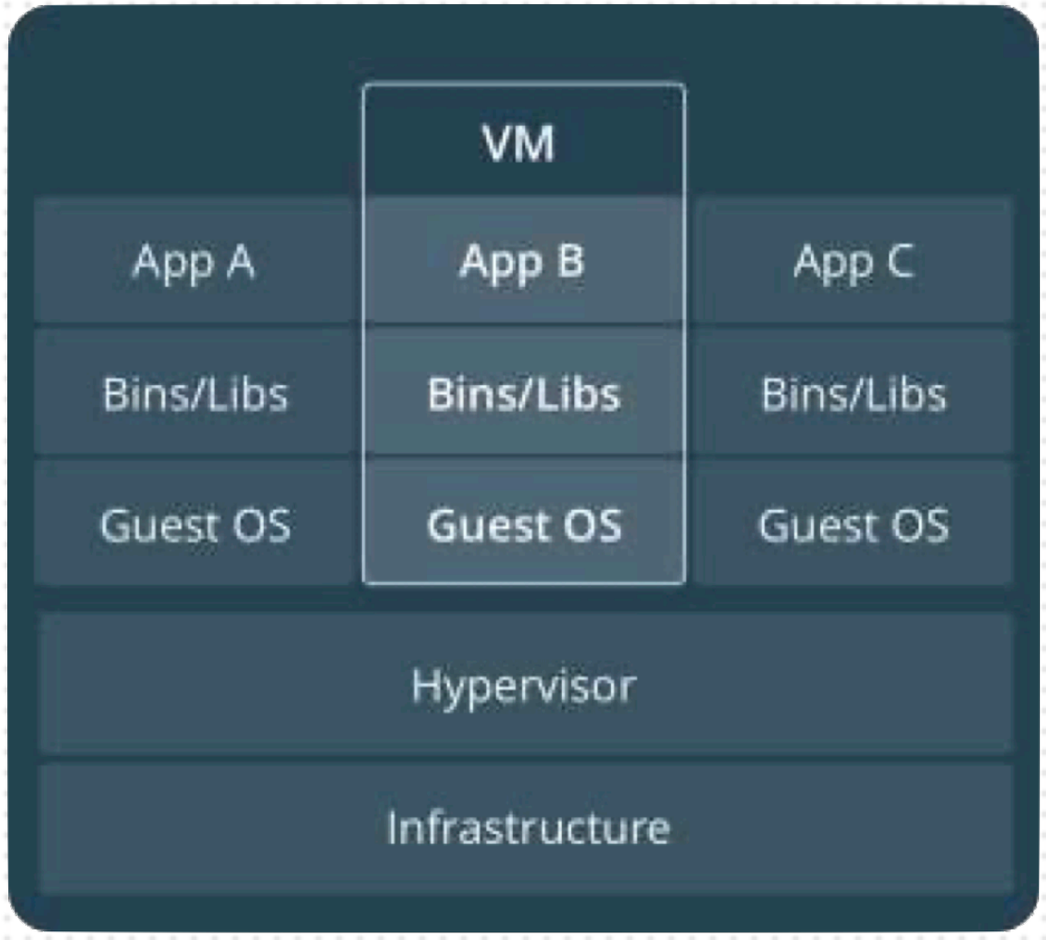
- Standardized packaging for software and dependencies
- Isolate apps from each other
- Share the same OS kernel
- Works with all major Linux and Windows Server



# Containers vs. VMs

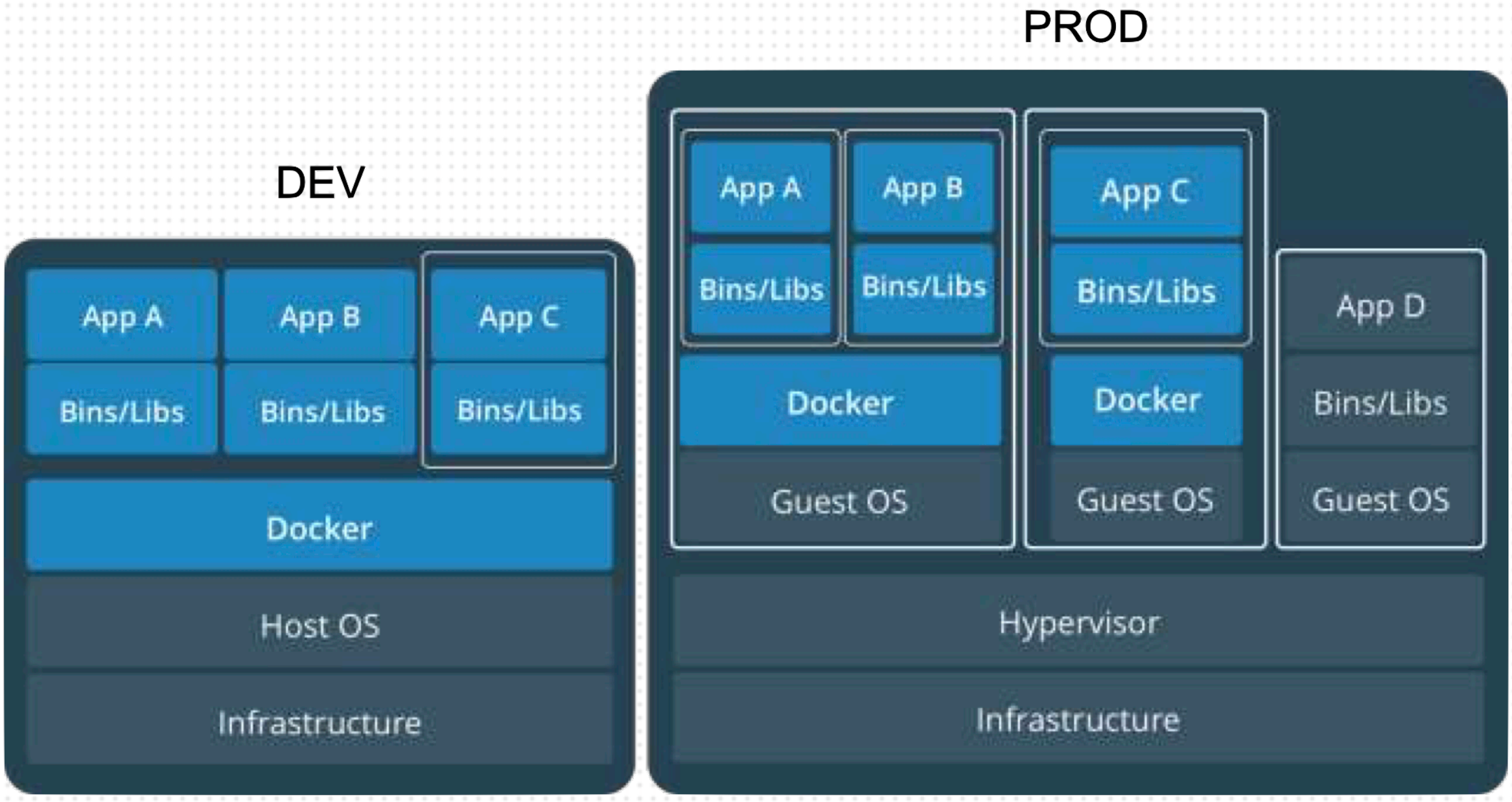



Containers are an app level construct




VMs are an infrastructure level construct to turn one machine into many servers

# Combining VMs and Containers



# Key Benefits of Docker Containers

## Speed

- No OS to boot = applications online in seconds

## Portability

- Less dependencies between process layers = ability to move between infrastructure

## Efficiency

- Less OS overhead
- Improved VM density

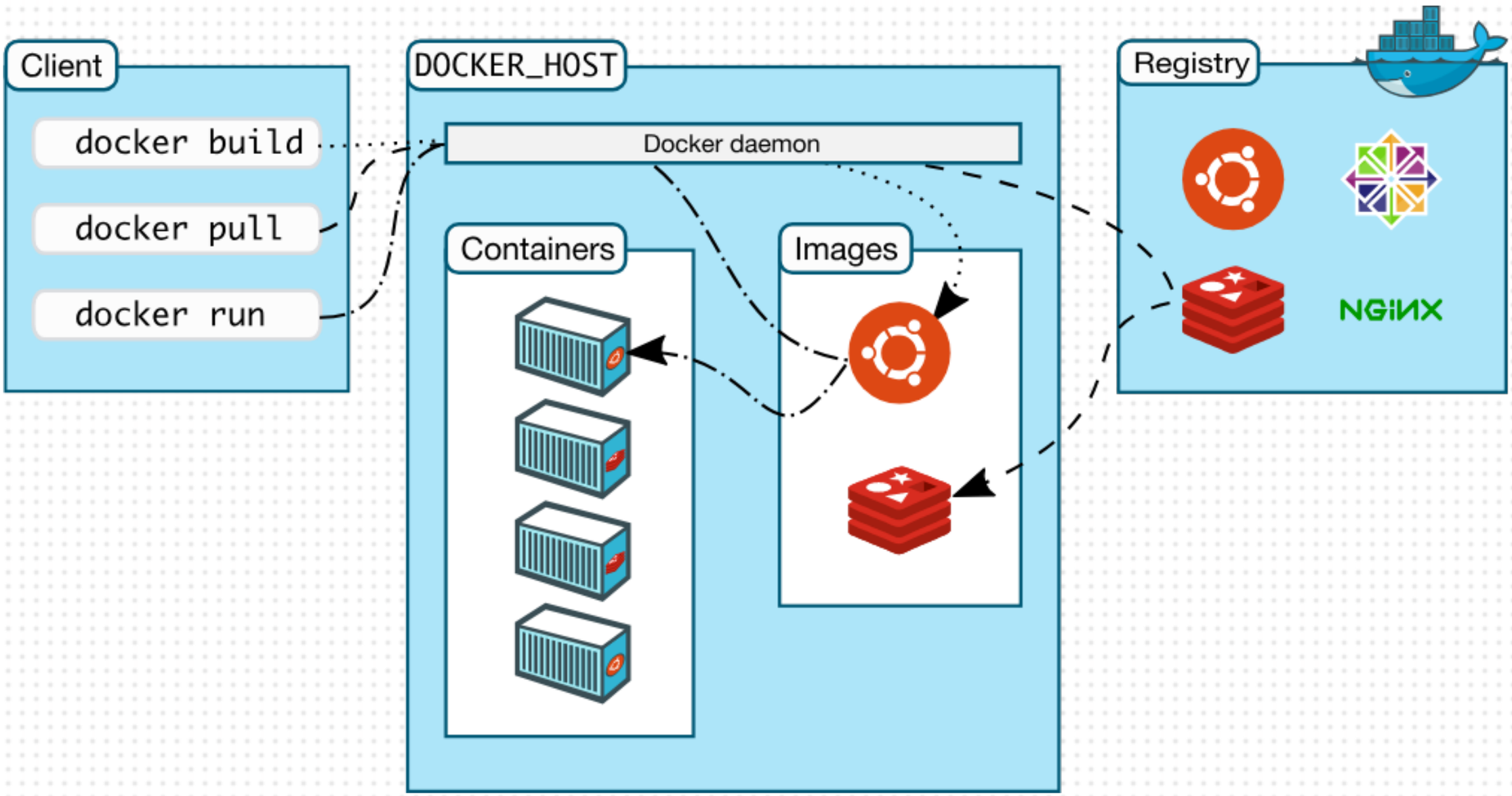
# Docker Basics

- **Images:** The blueprints of our application which form the basis of containers.
  - using “docker pull” command to download images
- **Containers:** Created from Docker images and run the actual application.
  - using “docker run” to run an image and “docker ps” for the list of running containers
- **Docker Daemon:** The background service running on the host that manages building, running and distributing Docker containers. The daemon is the process that runs in the operating system which clients talk to.

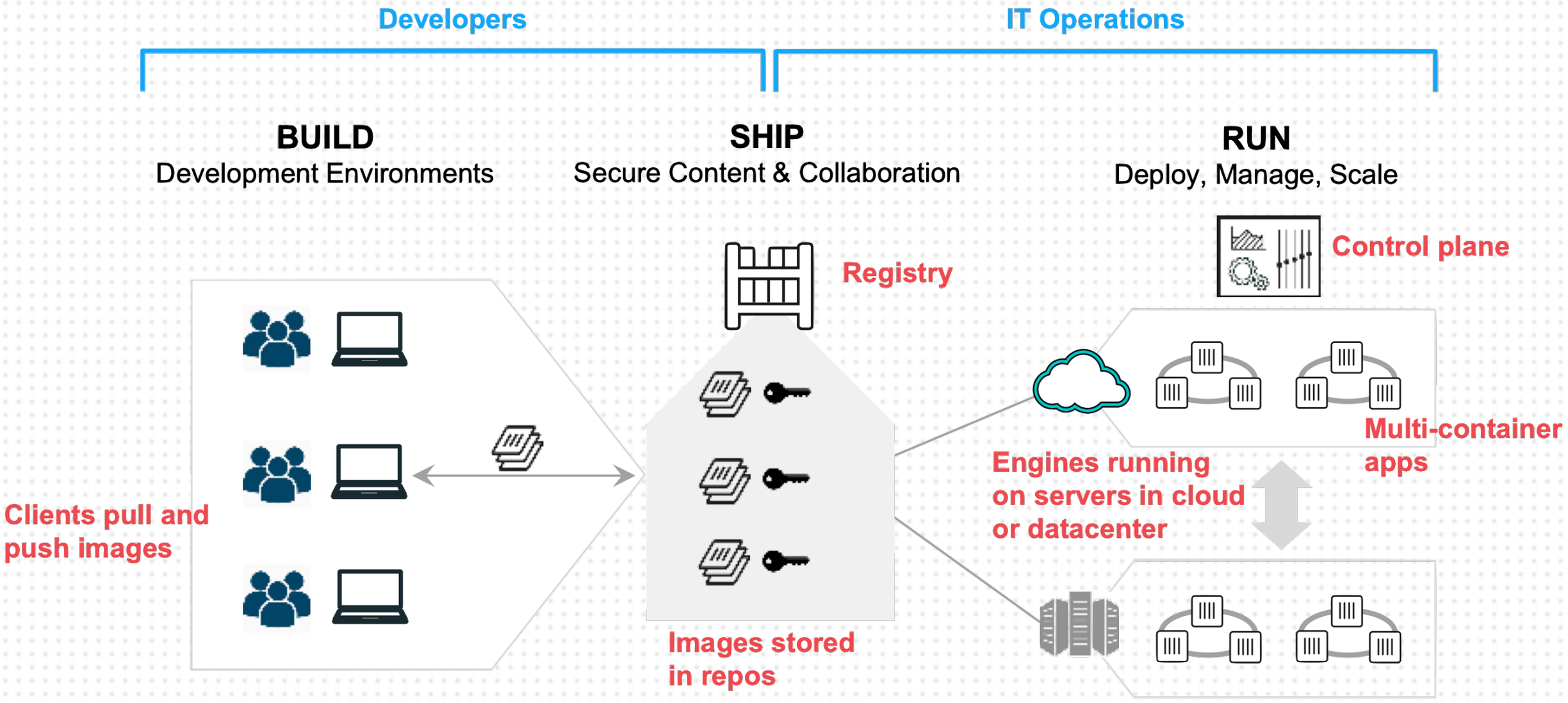
# Docker Basics

- **Docker Client:** The command line tool that allows the user to interact with the daemon.
  - Kitematic provides a GUI for commands
- **Docker Hub (Registry):** A registry of Docker images
  - A directory of all available Docker images
  - One can host their own Docker registries and can use them for pulling images.

# Docker Basics



# Container as a Service



# Building a Secure Supply Chain

## Container App Lifecycle Workflow

| | | |
|---|---|---|
| Private Image Registry | Secure Access and User Management | Application and Cluster Management |
| Image Scanning and Monitoring | Content Trust and Verification | Policy Management |

| | | |
|---|---|---|
| Security | Network | Volumes |
| Distributed State | Container Runtime | Orchestration |

Enterprise Edition

Docker Engine

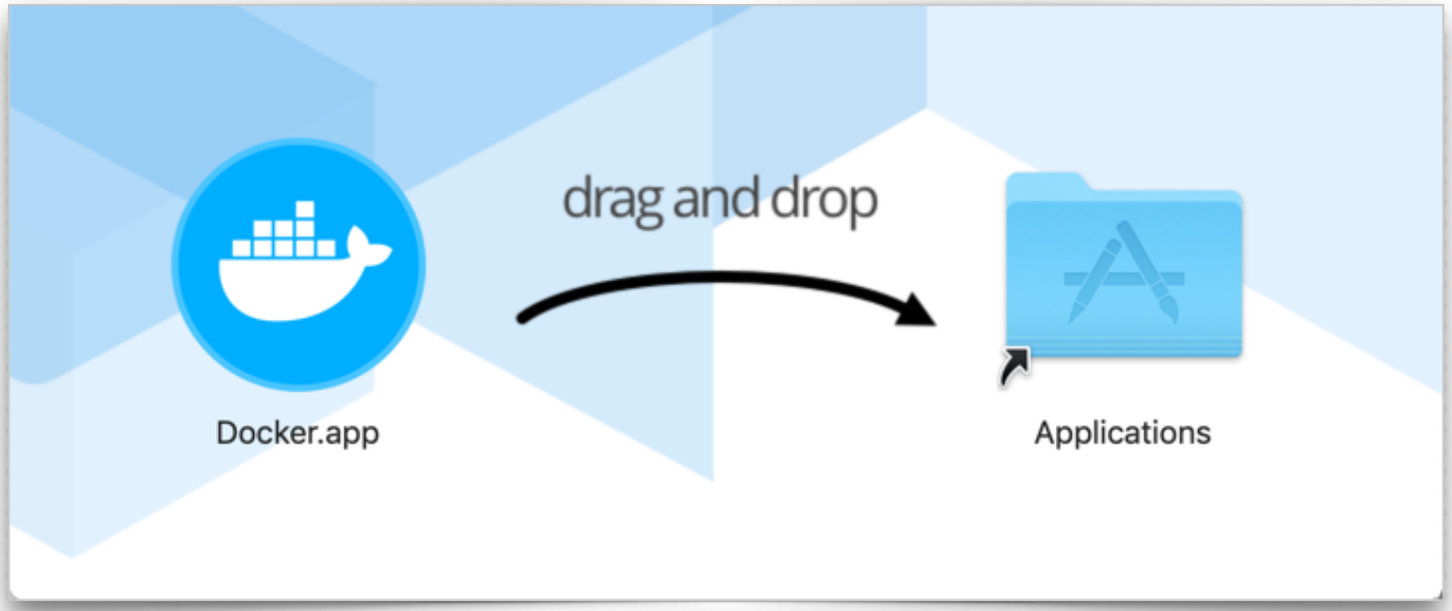
Usable Security

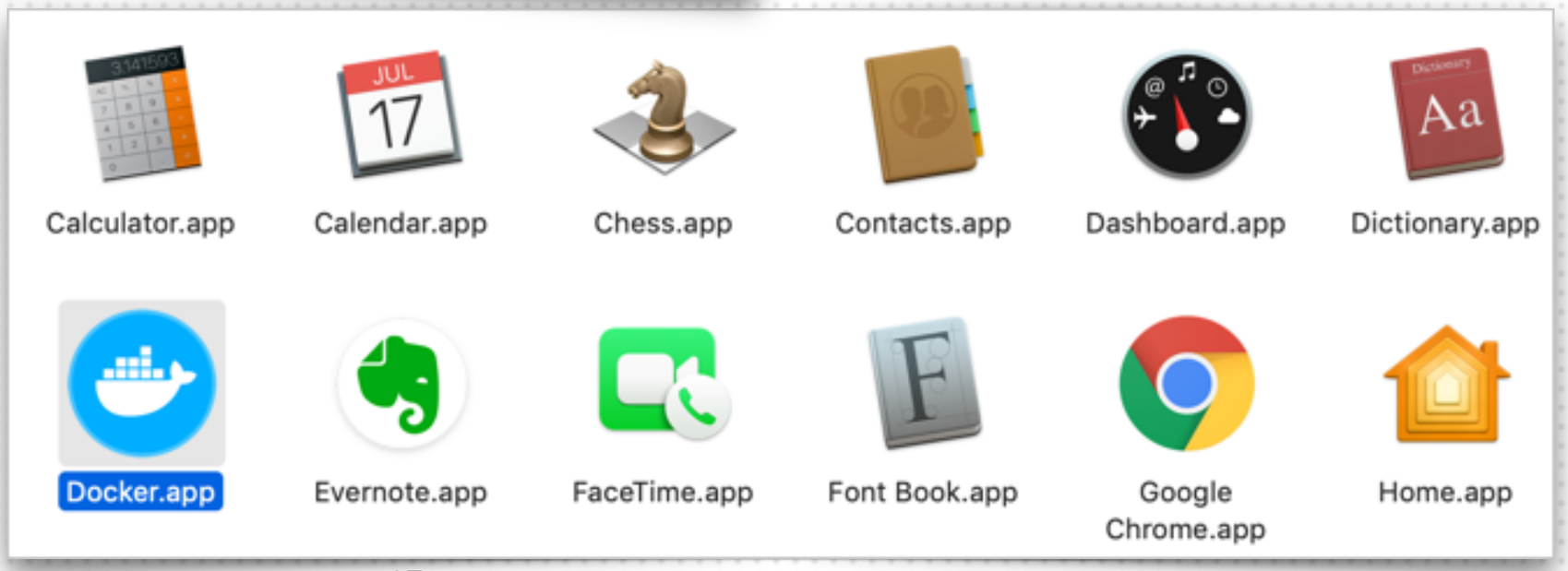
Trusted Delivery

Portable

# Getting Started

- Download Docker!



# Successful Installation

- Once you are done installing Docker, test your Docker installation by running the following:

```
$ docker run hello-world

Hello from Docker.
This message shows that your installation appears to be working correctly.
...
```

# Hello World!

- Pulling an image

```
$ docker pull busybox
```

```
$ docker images
REPOSITORY      TAG             IMAGE ID        CREATED         VIRTUAL SIZE
busybox         latest          c51f86c28340    4 weeks ago     1.109 MB
```

- Running a Docker container based on an image

```
$ docker run busybox
$
```

```
$ docker run busybox echo "hello from busybox"
hello from busybox
```

# Status of Containers

- Containers are fast because: imagine booting up a virtual machine, running a command and then killing it!
- What are containers that are currently running?

```
$ docker ps
CONTAINER ID    IMAGE          COMMAND       CREATED          STATUS
```

```
$ docker ps -a
CONTAINER ID    IMAGE          COMMAND       CREATED          STATUS
305297d7a235     busybox        "uptime"       11 minutes ago   Exited (0)
ff0a5c3750b9     busybox        "sh"           12 minutes ago   Exited (0)
14e5bd11d164     hello-world    "/hello"        2 minutes ago   Exited (0)
```

# Clearing Docker

- Removing by container ID

```
$ docker rm 305297d7a235 ff0a5c3750b9
305297d7a235
ff0a5c3750b9
```

- More Intelligence!

```
$ docker rm $(docker ps -a -q -f status=exited)
```

- The same effect using prune

```
$ docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
4a7f7eebae0f63178aff7eb0aa39f0627a203ab2df258c1a00b456cf20063

Total reclaimed space: 212 B
```

# Deploying Web Apps to Tomcat on Docker

- Developing a Docker file specification
- We write a Dockerfile specifying the base image to be used, the webapp to be used, etc.

```
FROM tomcat:8.0-alpine
LABEL maintainer="e.khamespanah@ut.ac.ir"

ADD sample.war /usr/local/tomcat/webapps/

EXPOSE 8090
CMD ["catalina.sh", "run"]
```

# Meaning of the Docker Instructions

- **FROM:** initializes a new build stage and sets the Base Image
- **LABEL:** sets the Author field of the generated images (any key-value)
- **ADD:** copies new files, directories or remote file to the filesystem of the image
  - We are adding sample.war in the folder /usr/local/tomcat/webapps/
  - From the Tomcat image documentation, we know that the default path of Tomcat is /usr/local/tomcat on the container.
- **EXPOSE:** informs Docker that the container listens on the specified network ports at runtime.
- **CMD:** specifies what to run when the container (not the image) is run.
  - There can only be one CMD instruction in a Dockerfile.
  - Don’t confuse RUN with CMD. RUN runs a command at build time.

# Building the Image

- Building an image from the content of the folder /home/…/allfiles and call it “mywebapp”

```
$ docker build -t mywebapp /home/…/allfiles

Sending build context to Docker daemon 60.42kB
Step 1/5 : FROM tomcat:8.0-alpine
8.0-alpine: Pulling from library/tomcat
…
Status: Downloaded newer image for tomcat:8.0-alpine
Step 2/5 : LABEL maintainer="deepak@softwareyoga.com"
Step 3/5 : ADD sample.war /usr/local/tomcat/webapps/
Step 4/5 : EXPOSE 8090
Step 5/5 : CMD ["catalina.sh", "run"]
Successfully built ab09b4cafc66
Successfully tagged mywebapp:latest
```

# Building the Image

- Note that all the files in the specified directory get “tar”ed and sent to the Docker daemon.
- Remember that the daemon could be running on a remote machine and that no parsing of the Dockerfile happens at the client side, so all the files at PATH get sent, not just the ones listed to ADD in the Dockerfile.

# Running the New Container

- Running and mapping the container port 8090 for the mywebapp image to the real port 8080 on the host machine:

```
$ docker run -p 8080:8090 mywebapp

30-Oct-2018 22:17:48.498 INFO [main]
org.apache.catalina.startup.VersionLoggerListener.log Server version: Apache
Tomcat/8.0.53
...
// LOGS cut for brevity
…
30-Oct-2018 22:17:48.691 INFO [main] org.apache.catalina.startup.Catalina.load
Initialization processed in 926 ms
…
30-Oct-2018 22:17:50.212 INFO [main] org.apache.catalina.startup.Catalina.start
Server startup in 1520 ms
```

# Running the New Container

